

Project Report

on

First Park Then Pay

By

Udhayarajan J

Under the Guidance of

Nidhi Damle

Master of Computer Application

P.E. Society's

Modern Institute of Business Studies (Autonomous)

Pune

2025-2026



Progressive Education Society's
Modern Institute of Business Studies (Autonomous)

Certificate

This is to certify that the Mini Project (MMPRJ-2511) entitled **First Park Then Pay** is submitted by **UDHAYARAJAN J** a Bonafide student of this institute, studying for a Master of Computer Application, has successfully carried out MCA semester-I Mini Project. This project report is submitted in partial fulfilment of **M.C.A. Semester I Academic Year 2025-26** curriculum as per Institute norms.

Dr. Maithili Arjunwadkar
Director
PES MIBS

Project Guide

Internal Examiner

ACKNOWLEDGEMENT

It is my proud privilege to express my profound gratitude to the entire management of Progressive Education Society's Modern Institute of Business Studies (Autonomous) and Faculty members of the institute for providing me with the opportunity to avail the excellent facilities and infrastructure of the institute. The knowledge and values inculcated have proved to be of immense help to my career.

I am grateful to **Prof. Dr. Maithili Arjunwadkar (Director, MIBS)** for their astute guidance, constant encouragement and sincere support for this project work.

I express my sincere gratitude to **Nidhi Damle (Assistant Professor, MCA Department, MIBS)** for their inspiration, constructive suggestion, mastermind analysis and affectionate guidance in my work, without which this project work completion would have been impossible for me. Sincere thanks to all my seniors and colleagues at the company for their support and assistance throughout the project.

I would like to thank all the faculties for providing me with an opportunity to pursue my industrial training, as it is an important part of the MCA course.

I feel proud and privileged in expressing my deep sense of gratitude to all those who have helped me in presenting this assignment. I would be failing in my endeavour if I do not place my acknowledgement.

Index

Chapter	Description	Page Number
1	INTRODUCTION	
1.1	Problem Statement	1
1.2	Objectives of the project	3
1.3	Scope of Work	4
1.4	Detail Description of Technology Used	6
1.5	System Overview	8
2	PROPOSED SYSTEMS	
2.1	Existing System	9
2.2	Limitation of Existing System	11
2.3	Propose System	13
3	SYSTEM ANALYSIS	
3.1	Functional Requirements	15
3.2	Non-Functional Requirements	17
3.3	Feasibility Study	19
3.4	Use Case Diagram	21
3.5	System Architecture	23
4	SYSTEM DESIGN	
4.1	ER Diagram	26
4.2	User Interface Design	30
5	IMPLEMENTATION	
5.1	Coding Environment	36
5.2	Module Description	40
6	CONCLUSION	
6.1	Conclusion	43
6.2	Future Scope	45
7	BIBLIOGRAPHY	
7.1	References	48

Chapter 1 – Introduction

1.1 Problem Statement

Managing parking spaces in urban areas has become increasingly difficult due to the continuous growth in the number of vehicles. Traditional parking systems still rely on manual processes such as physical tokens, handwritten logs, and human supervision. These outdated practices significantly reduce efficiency and often lead to operational problems. Users experience delays while searching for free parking slots, and administrators struggle to maintain accurate information on vehicle entries, exits, payments, and slot status. As a result, the overall parking experience becomes time-consuming, error-prone, and inconvenient.

A major challenge with manual parking management is the **lack of real-time visibility**. Without a digital system, users do not know whether a slot is available until they physically reach the parking area. This leads to congestion, unnecessary vehicle movement, and frustration. Administrators also face difficulties in tracking occupancy, monitoring revenue, preventing unauthorized parking, and resolving disputes because there is no centralized data record.

Furthermore, manual billing creates **transparency issues**, such as incorrect fee calculation, human errors, and difficulty in verifying parking duration. As cities continue to grow and parking spaces become limited, it is essential to adopt an automated, scalable, and efficient solution that can streamline operations, reduce human dependency, and improve user satisfaction.

Key Issues Identified

- **No real-time slot availability**, causing users to waste time searching for free spaces.
- **Manual entry/exit management**, leading to delays and frequent mistakes.
- **Lack of transparency in billing** and difficulty verifying the parking duration.
- **Inefficient tracking of vehicles**, slot occupancy, and payment status.
- **Difficulty in generating reports** due to lack of centralized digital data.
- **Heavy dependency on manual labour**, increasing chances of human error.

A modern digital parking management system is required to overcome these issues and deliver a seamless, transparent, and automated parking experience.

1.2 Objectives of the Project

The primary objective of this project is to design and implement a fully automated digital parking management system that improves the efficiency, speed, and accuracy of the overall parking process. The system aims to provide users with real-time information about parking slot availability so they can plan their parking without confusion or delays. By adopting a “park first, pay later” model, the system ensures transparent billing based on the actual parking duration.

Security is one of the main focus areas of the project. Therefore, **Key cloak** is integrated for secure authentication, authorization, and role-based access control. This ensures that only verified users and administrators can access the system. All critical data—including slots, vehicles, users, bills, and payment history—is stored in a centralized database, making it easy to generate reports and maintain accurate records over time.

A **Spring Boot REST API-based backend** ensures scalability and modular development, while **Docker** is used for deploying the system in a consistent and portable manner across any environment. The system also includes admin dashboards that offer insights into parking utilization, payment completion rate, and vehicle count. These dashboards help administrators make informed decisions and manage the parking facility more effectively.

Main Objectives

- **Provide real-time parking slot availability** to reduce waiting time for users.
- **Enable “park first, pay later” billing**, ensuring transparency and accuracy.
- **Integrate Key cloak for secure authentication and role-based access control.**
- **Maintain centralized data storage** for users, slots, payments, and history.
- **Build a scalable backend using Spring Boot REST APIs.**
- **Deploy the application using Docker** for portability and reliability.
- **Offer admin dashboards** to monitor vehicle count, slot usage, and revenue.

Through these objectives, the project ensures a seamless experience for both users and administrators while maintaining high levels of security and efficiency.

1.3 Scope of the Project

The scope of this project includes developing a complete web-based parking management system suitable for small, medium, and large parking providers. The system offers secure login, role-based access for users and administrators, and real-time management of parking slots. Each slot can have a status such as available, booked, or occupied, enabling users and admins to monitor the lot efficiently.

The system also supports configuration of parking slots based on **vehicle type** (two-wheeler, four-wheeler, etc.), ensuring proper allocation and maximizing space utilization. Billing is calculated automatically based on the duration of the parking session, and payment status is tracked accurately as paid or pending. Administrators can generate different types of reports and monitor trends through dashboards that display insights such as total vehicles, peak hours, slot usage percentage, and total revenue generated.

The current scope focuses on delivering a functional and efficient system using web technologies and backend services but excludes high-tech integrations like IoT sensors and mobile apps, which are part of the future enhancements.

In Scope (Current Features)

- **Secure login and role-based authorization** using Key cloak.
- **Real-time parking slot management** (Available, Booked, Occupied).
- **Vehicle type and slot configuration** to optimize space utilization.
- **Automatic billing based on parking duration.**
- **Payment tracking** with clear paid/pending status.
- **Admin dashboards and detailed reports.**
- **Docker-based deployment** to ensure consistency across environments.

Out of Scope (Future Enhancements)

- **IoT sensor integration** for automatic detection of empty slots.
- **Dedicated mobile app** for customers and administrators.
- **Online payment gateway integration** (UPI, credit/debit cards).
- **AI-based parking prediction** to forecast demand and availability.

These elements clearly define what will be delivered in the current project and what enhancements can be implemented in the future.

1.4 Technologies Used

This project uses a modern, scalable, and secure technology stack to ensure smooth performance, maintainability, and long-term growth. The frontend is developed using **HTML5, CSS3, and JavaScript**, with the option to use **Angular** for building a more interactive, component-driven user interface. These technologies provide responsive and user-friendly screens that allow customers to easily check slot availability, register themselves, and manage parking-related tasks.

On the backend side, the project uses **Java 21** along with the **Spring Boot** framework to create lightweight, modular, and high-performance REST APIs. Spring Boot simplifies development by providing features like dependency injection, auto-configuration, and built-in support for REST controllers, making the backend scalable and reliable.

For authentication and authorization, **Keycloak** is integrated to provide secure user login and role-based access control. It manages user sessions, tokens, and permissions, ensuring that normal users and administrators have separate and secure access.

Data is stored using **MySQL 8.0**, a powerful relational database with strong performance, indexing, and data integrity. MySQL ensures reliable storage of user details, slot information, parking history, and payment records.

From a DevOps perspective, **Docker** is used to containerize all services, ensuring consistency across different environments such as development, testing, and production. Version control is handled using **Git and GitHub**, enabling collaborative development, change tracking, and continuous updates. For testing and API validation, **Postman** is used to simulate requests and verify that system workflows function as expected. The entire build lifecycle is managed using **Gradle**, which automates compiling, testing, and packaging the application.

Technologies Used

- **Frontend:** HTML5, CSS3, JavaScript / Angular
- **Backend:** Java 21, Spring Boot
- **Authentication:** Key cloak
- **Database:** MySQL 8.0
- **DevOps:** Docker, Git, GitHub

- **Testing Tools:** Postman
- **Build Tool:** Gradle

This combination of technologies ensures the system is modern, secure, scalable, and easy to deploy across different platforms.

1.5 System Overview

The parking management system is designed as a complete digital solution that simplifies the parking workflow for both customers and administrators. The system allows customers to view real-time parking slot availability directly from the interface. Users can select their vehicle type, choose an available slot, park their vehicle, and complete the payment after leaving the premises. This “park first, pay later” approach provides a transparent and user-friendly experience.

Administrators have access to an advanced backend panel where they can configure parking-related data such as vehicle types, block names, and the number of slots in each block. Admins can also manage parked vehicles, monitor occupancy, verify payments, and generate usage reports. The admin dashboard provides a complete overview of daily activity, helping administrators make informed decisions to optimize parking operations.

The entire backend runs on **RESTful APIs** built with Spring Boot, ensuring fast response times and clean separation between the client and server. All user and admin requests are secured through **Key cloak**, which provides token-based authentication using OAuth2 and OpenID Connect. This ensures that sensitive operations, such as payment updates and slot management, are accessible only to authorized users.

To ensure consistency and easy deployment across different machines, the system runs inside **Docker containers**. The backend service, Key cloak server, and MySQL database each operate in separate containers, making the system modular and easier to scale. This containerized environment also supports future enhancements such as load balancing and microservices.

Overall, the system provides a reliable and modern platform that makes parking operations more efficient, reduces manual effort, and ensures secure and centralized control over all parking-related activities.

Chapter 2 – Literature Review / Existing System

2.1 Existing System

Traditional parking management systems still rely on manual processes and human supervision, which makes the overall workflow slow, inefficient, and highly prone to errors. In the existing system, parking attendants usually maintain handwritten logs or paper registers to record vehicle entry and exit times. This manual approach often leads to mismatches in data, difficulty in tracking vehicles, and challenges in verifying the exact duration of parking. Since the entire process depends heavily on human involvement, even a small mistake can result in incorrect billing or disputes between the user and the administration.

Another major limitation of the existing system is the **absence of real-time slot availability**. Users have no way of knowing whether parking spots are available until they physically reach the location. This often leads to congestion at parking entrances, unnecessary vehicle movement inside the parking area, and frustration among customers, especially during peak hours. Without digital tracking, administrators cannot identify which slots are free, booked, or occupied at any given moment.

Billing in traditional parking systems is also calculated manually. Attendants write down entry and exit times and then compute charges based on a pre-defined rate chart. Manual calculations increase the chances of errors such as overcharging, undercharging, or miscalculating the total amount. There is also no proper system to track payments, leading to transparency issues and difficulty in auditing.

Paper-based receipts further create management problems. They can be lost, damaged, or misplaced, making it difficult to verify transactions or maintain a proper history of parking activities. Since there is no centralized database, administrators face challenges in preparing reports, analysing usage patterns, or monitoring revenue.

These drawbacks highlight that the existing system is outdated and unable to meet the needs of modern, high-traffic parking environments. A digital, automated, and real-time solution is necessary to overcome these limitations and improve both user convenience and administrative efficiency.

Limitations of the Existing System

- **Manual entry and exit recording**, increasing the chances of human error.
- **Paper-based receipts**, which are easy to lose or damage.
- **No real-time availability tracking**, leading to congestion and confusion.
- **Manual billing calculations**, often resulting in inaccuracies.
- **High chance of errors** due to complete dependence on manual processes.

2.2 Limitations of Existing System

The traditional parking management systems suffer from several limitations that directly affect operational efficiency, user satisfaction, and the overall reliability of the parking process. Since these systems rely heavily on manual work, they are slow, inconsistent, and lack the accuracy needed to manage high volumes of vehicles in busy urban areas. The dependence on human involvement for tasks such as recording vehicle entries, issuing receipts, calculating bills, and tracking payments creates multiple opportunities for errors and delays.

One of the most significant limitations is the **time-consuming nature** of manual processes. Attendants must individually register each vehicle, note timings, calculate charges, and update records. During peak hours, this leads to long queues at entry and exit points, causing inconvenience for both users and staff. Because the process is not automated, human errors such as noting incorrect times, misplacing receipts, or miscalculating fees are very common. These issues often result in disputes and undermine trust in the system.

Another challenge is the **lack of transparency** in billing. Without a digital system to automatically record and compute parking duration, customers may feel unsure whether they are being charged correctly. Manual billing does not provide detailed breakdowns or error-free calculations, which reduces confidence in the parking service. The absence of centralized tracking also makes auditing difficult for administrators.

The traditional system also leads to **congestion**, as users have no advance information about available parking slots. Drivers must roam the parking area in search of a free space, wasting fuel and contributing to traffic delays. This not only affects user convenience but also increases the workload on parking staff who must manually guide vehicles.

Monitoring capabilities in the existing system are limited as administrators cannot view real-time data. They rely on physical registers or verbal updates from staff, making it difficult to analyze parking usage, identify trends, or detect overcrowding. This prevents effective decision-making and hinders improvement in operations.

Additionally, traditional systems lack **scalability**. As the number of vehicles increases or the parking area expands, the manual approach becomes even harder to manage. There is no

mechanism to handle large-scale operations or integrate modern technologies such as digital payments, IoT sensors, or automated reporting.

These limitations clearly show the need for an automated, real-time, and transparent parking management solution.

Limitations of the Existing System

- **Time-consuming and prone to human error**, affecting accuracy and speed.
- **No transparency in billing**, leading to doubts and disputes.
- **Congestion due to manual processes**, causing long queues and delays.
- **Limited monitoring capability**, with no real-time visibility.
- **Poor scalability**, making it unsuitable for large or expanding parking facilities.

2.3 Proposed System

The proposed “**First Park Then Pay**” system introduces a modernized, fully automated approach to parking management, addressing the inefficiencies and limitations found in traditional systems. This digital solution transforms the entire workflow by eliminating manual processes and introducing real-time monitoring, secure authentication, automated billing, and centralized data management. With this system, both users and administrators experience a more streamlined, transparent, and efficient parking process.

The core feature of the proposed system is **digital slot tracking**, which enables users to view the live availability of parking spaces before entering the area. This significantly reduces congestion, as users no longer need to drive around searching for empty slots. The system dynamically updates slot statuses such as Available, Occupied, or Booked, ensuring accurate real-time visibility for users and administrators.

Another major improvement is **automated billing**. Instead of relying on manual calculations, the system automatically records the vehicle’s entry and exit times and calculates the parking fee based on predefined rules. This not only improves accuracy but also ensures complete transparency. Users can clearly see how much they are being charged and why, reducing disputes and improving trust in the parking service.

To secure user access and ensure proper role management, the proposed system incorporates **secure user login** through Key cloak. This authentication mechanism ensures that only authorized users and administrators can access specific features. Regular customers can book or park vehicles, while admins can manage slots, payments, and overall system configurations.

For administrators, the system provides a **dashboard-based monitoring interface**. This dashboard displays key information such as the number of available slots, occupied slots, ongoing parking sessions, pending payments, and overall vehicle flow. It allows admins to make faster decisions, detect peak hours, track revenue, and manage operations more effectively.

Additionally, the system supports **real-time status updates**, meaning any changes—such as a vehicle entering, exiting, or payment being completed—are immediately reflected across all

platforms. This ensures data accuracy, improves operational control, and enhances user experience.

Overall, the proposed system delivers a modern, automated, and scalable parking management platform that eliminates manual errors, enhances transparency, and significantly improves the efficiency of parking operations. It lays the foundation for integrating future enhancements such as digital payments, IoT sensors, and mobile applications.

Key Features of the Proposed System

- **Digital slot tracking** for accurate real-time visibility.
- **Automated billing** with precise time-based calculation.
- **Secure user login** supported by Key cloak authentication.
- **Dashboard-based monitoring** for administrators.
- **Real-time status updates**, ensuring accurate and instant information.

Chapter 3 – System Analysis

3.1 Functional Requirements

The functional requirements define how the proposed parking management system should behave and outline the key operations that both users and administrators must be able to perform. These requirements describe the essential actions needed to ensure smooth, secure, and transparent functioning of the “First Park Then Pay” system. They ensure that the system meets user expectations while providing administrators full control over parking configurations, payment management, and real-time monitoring.

From the user’s perspective, the system must provide a simple, intuitive, and secure interface through which they can perform all necessary parking activities. The **login and authentication** requirement ensures that each user is uniquely identified and securely verified before accessing the system. This prevents unauthorized access and protects user data. Once logged in, users must be able to **view available parking slots**, allowing them to make informed decisions before arriving at the parking location. The system should display real-time updates on slot availability to reduce congestion and confusion.

Users must also have the ability to **select their vehicle type**, ensuring they are directed to the correct slot based on vehicle category (e.g., two-wheeler, four-wheeler). After selecting a slot, users should be able to **park their vehicle**, and the system must automatically record the start time of their parking session. Once the parking duration ends, the system should enable users to **pay after parking**, ensuring transparent, automated, time-based billing. Finally, users should be able to **view their parking history**, which includes past sessions, timings, and payment records—helping them keep track of their usage and expenses.

For administrators, the system must offer powerful management features that allow them to monitor and control the entire parking environment. Admins should be able to **manage vehicle types**, adding or modifying categories as needed. They must also be able to **create blocks and slot numbers**, organizing the parking area into manageable sections with clearly defined slots. Real-time visibility is essential, so administrators must be able to **view parking activity**, including active sessions, occupied slots, and pending payments.

Payment handling is a critical function, and administrators must be able to **manage payments** by reviewing pending bills, confirming completed transactions, and generating reports. Additionally, the system should provide access to **admin dashboards**, offering graphical insights into slot usage, vehicle flow, total revenue, and system performance. These dashboards help administrators make data-driven decisions and improve operational efficiency.

Together, these functional requirements ensure that the system supports seamless parking operations, offers transparency to users, and provides administrators with the necessary tools to manage and monitor the entire environment effectively.

User Functional Requirements

- **Login & authentication** to ensure secure access.
- **View available parking slots** in real time.
- **Select vehicle type** based on category.
- **Park vehicle** and start the session automatically.
- **Pay after parking duration** using system-generated billing.
- **View parking history** including past visits and payments.

Admin Functional Requirements

- **Manage vehicle types** (add, update, delete categories).
- **Create blocks and slot numbers** for structured parking layout.
- **View parking activity** in real time.
- **Manage payments** and verify completed transactions.
- **Access dashboards** for reports, monitoring, and analysis.

3.2 Non-Functional Requirements

Non-functional requirements describe the quality attributes of the parking management system and define how well the system should perform under various conditions. These requirements ensure that the solution is not just functional but also secure, efficient, user-friendly, reliable, and capable of supporting future growth. They play a crucial role in shaping the overall user experience, system architecture, and long-term sustainability of the application.

Security is one of the most critical non-functional aspects of the system. Since the application manages sensitive data such as user information, login credentials, and payment details, it must provide strong protection mechanisms. The system uses **token-based authentication**, powered by Key cloak, to ensure that only authenticated users and administrators can access protected resources. Additionally, the architecture is designed to be **HTTPS-ready**, meaning the application can be deployed with SSL certificates to encrypt all communication between the client and server. This protects the system from common attacks such as data interception, unauthorized access, and session hijacking.

Performance is another essential requirement, especially because the system provides real-time slot updates and handles multiple user requests simultaneously. The application must deliver **fast loading times**, responsive APIs, and efficient database interactions to ensure a smooth parking experience for users. Real-time updates, such as slot availability and status changes, must be reflected without delay to help users make quick decisions and reduce congestion at parking locations. Optimized backend logic, database indexing, and containerized deployment contribute to the overall performance and responsiveness of the system.

Usability ensures that users of all experience levels can easily interact with the system. A **simple UI and intuitive navigation** structure are essential to help users check slot availability, initiate parking, and complete payments without confusion. The interface must be clean, accessible, and logically organized. Admin panels should also follow usability best practices, providing clear dashboards and simple forms so that administrators can manage data effortlessly.

Reliability is equally important because the system must exhibit **consistent behaviour** during normal operations as well as peak usage conditions. The system should handle multiple requests, maintain accurate slot information, and process payments without failure. Reliability

ensures that both users and administrators can trust the platform for real-time parking operations. Proper error handling, data validation, and consistent session management all contribute to improving system reliability.

Finally, scalability ensures that the system can grow along with increasing users, vehicles, and parking slots. As parking facilities expand or as more users adopt the system, it should be able to handle the higher load without performance degradation. Containerized deployment using Docker, modular backend architecture, and an optimized database design allow the system to scale efficiently. This ensures that the application remains future-proof and can easily support additional modules such as mobile apps, IoT integrations, or advanced reporting.

Non-Functional Requirements Summary

- **Security:** Token-based authentication and HTTPS-ready design for secure data transmission.
- **Performance:** Fast loading, optimized APIs, and real-time updates for smooth operation.
- **Usability:** Simple and intuitive user interface for both users and admins.
- **Reliability:** Consistent behaviour, accurate updates, and stable performance.
- **Scalability:** Ability to support increasing numbers of users, vehicles, and parking slots.

3.3 Feasibility Study

A feasibility study evaluates whether the proposed Smart Parking Management System can be developed and implemented successfully within the available constraints. The system is analysed from three major perspectives—**economic**, **technical**, and **operational**—to ensure its practicality and long-term sustainability.

Economic Feasibility

Economic feasibility examines whether the system is financially viable and cost-effective. The Smart Parking System requires only basic infrastructure such as a server environment for hosting the backend, a front-end user interface, and a relational database like MySQL. Since the project uses open-source technologies and frameworks, the overall development cost is minimal. There is no need for expensive hardware or proprietary software licenses, reducing upfront investment. The system also has the potential to reduce manual labour and operational expenses by automating slot booking, payments, and record-keeping. Therefore, the project is economically practical and can be implemented even by small organizations or institutions with limited budgets.

Technical Feasibility

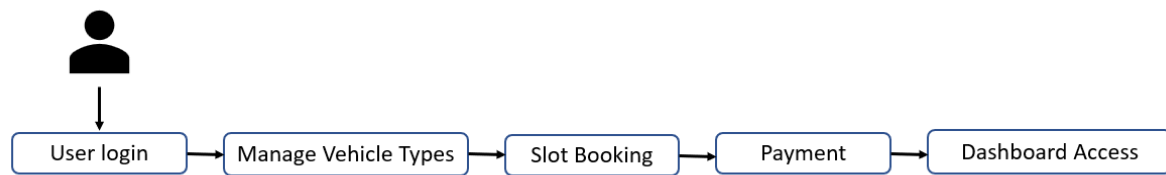
Technical feasibility evaluates whether the required technology stack and expertise are available. The system is designed using well-established technologies such as **Spring Boot**, **React/Angular (optional)**, **MySQL**, and **REST APIs**. These technologies are widely used, stable, and supported by large developer communities, ensuring long-term usability and ease of maintenance. The development tools required—such as IntelliJ/Eclipse, Postman, and MySQL Workbench—are easily accessible. The hardware requirements are minimal, allowing the system to operate smoothly on standard servers or cloud platforms. Since the development team is familiar with the selected technologies, the system is technically feasible and can be implemented without significant risks.

Operational Feasibility

Operational feasibility assesses how effectively the system can be used and managed after deployment. The Smart Parking System is designed with a simple dashboard for admins and an intuitive interface for users, ensuring smooth day-to-day operations. Users can easily search

for parking slots, book them, pay for their parking duration, and view their history. Administrators can manage slot availability, vehicle types, payments, and activity logs through a centralized dashboard. Since the system reduces manual supervision and offers automated workflows, it improves operational efficiency. Staff training requirements are minimal, and the system can be adopted quickly in real-world parking environments. Therefore, operational feasibility is high.

3.4 Use Case Diagram



The diagram represents the overall workflow of the Smart Parking Management System, showing how a user progresses through different modules of the application. Each stage demonstrates a critical function that ensures smooth parking operations, from login to dashboard access. The process is designed to be simple, intuitive, and efficient for both users and administrators.

1. User Login

The workflow begins with users accessing the system through a secure login interface. They must enter valid credentials to authenticate their identity. This ensures that only authorized users and administrators can access the system. Token-based authentication strengthens security by preventing unauthorized access and session hijacking.

2. Manage Vehicle Types

After logging in, users or administrators can manage vehicle-related information. This module allows administrators to define vehicle categories such as two-wheelers, four-wheelers, electric vehicles, etc. It ensures that the system recognizes different vehicle types and allocates appropriate parking slots. In some cases, users may select the type of vehicle they are parking, which influences slot availability and pricing.

3. Slot Booking

Once vehicle type management is complete, the next step is booking a parking slot. Users can view the available slots in real-time, check slot location, and reserve a slot for their vehicle. The system automatically updates slot availability to prevent double booking. This module improves convenience and reduces the time spent searching for parking spaces.

4. Payment

After parking or booking, the user proceeds to the payment module. Charges are calculated based on the parking duration and vehicle type. The system supports secure digital payments, ensuring a smooth and transparent transaction process. Payment records are stored for future reference and reporting.

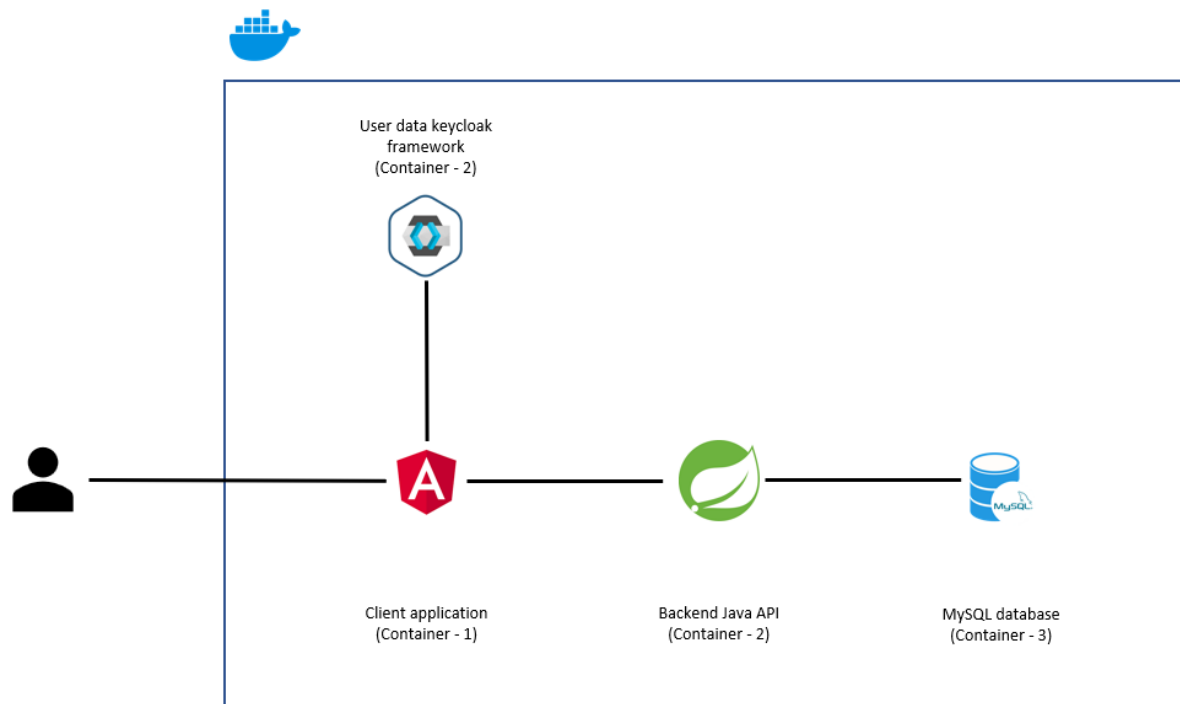
5. Dashboard Access

Finally, both users and administrators can access a dashboard.

- **Users** can view booking history, payment details, and current parking status.
- **Administrators** can monitor system activity, view reports, track payments, and manage operations effectively.

The dashboard provides real-time insights and serves as the control centre for the entire parking system.

3.5 System Architecture



The system follows a modern **three-tier architecture** combined with a dedicated security layer and container-based deployment. This architecture ensures high performance, modularity, and scalability while maintaining strict separation of concerns. Each layer performs a specific role in request handling, data processing, security, and storage. The entire solution runs inside Docker containers, making the system portable, consistent, and easy to deploy on any environment.

1. Presentation Layer (Client Application – Angular)

The presentation layer consists of the Angular-based frontend application, which runs inside **Container 1**. Users interact with the system through a browser interface where they can log in, view parking slot availability, book slots, and make payments. The frontend handles UI rendering, input validation, and communication with the backend REST services. It ensures a smooth and responsive user experience by providing real-time updates and intuitive navigation.

2. Security Layer (Key cloak Authentication Framework)

Authentication and authorization are managed by **Key cloak**, running inside **Container 2**. When users attempt to log in, the Angular application redirects them to Key cloak for secure user validation. Key cloak issues **JWT tokens** upon successful authentication, which are used for secure access to backend APIs.

This layer ensures:

- Role-based access
- Centralized user management
- Strong token-based authentication
- Session and identity management

By isolating security in a separate container, the system becomes more secure and maintainable.

3. Application Layer (Backend Java API – Spring Boot)

The backend is built using Spring Boot and runs inside **Container 3**. It serves as the core of the system, processing business logic and responding to frontend requests. The APIs handle operations such as parking slot management, vehicle type configuration, payment calculation, booking workflows, and dashboard analytics.

Key functionalities of this layer include:

- Handling REST requests from the Angular UI
- Applying business logic and validation
- Communicating securely with Key cloak for token verification
- Interacting with the MySQL database

This layer ensures fast, scalable, and reliable data processing.

4. Data Layer (MySQL Database)

The data layer stores all persistent information such as users, vehicle types, parking slots, bookings, and payment records. MySQL runs inside **Container 4**, providing structured relational storage. The backend APIs communicate with the database using optimized queries, ensuring data consistency, reliability, and security. MySQL supports ACID properties, enabling accurate logging of all parking transactions and system activities.

5. Deployment Layer (Docker Containers)

The entire system is deployed using **Docker**, with each component running in a separate container:

- **Container 1:** Angular UI
- **Container 2:** Key cloak
- **Container 3:** Spring Boot Backend
- **Container 4:** MySQL Database

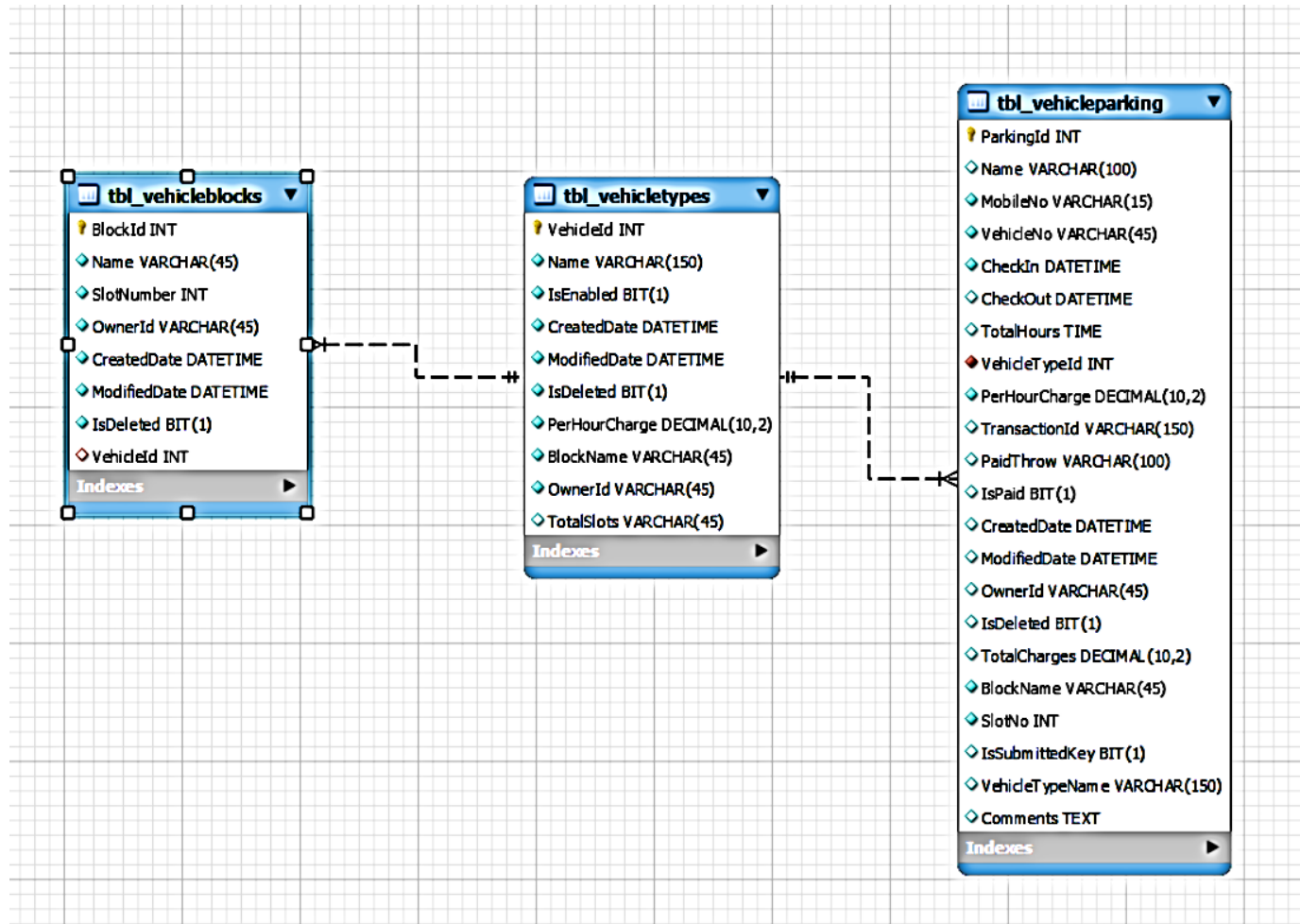
Docker ensures isolated, reproducible environments, simplifies configuration, and supports easy scaling. This setup is ideal for modern cloud-ready applications.

Overall Architecture Summary

The architecture integrates UI, backend logic, secure authentication, and database storage into a clean and modular framework. The dedicated security layer with Keycloak adds strong protection, while Docker-based deployment makes the system portable and efficient. This multi-layered approach ensures the application is maintainable, scalable, secure, and ready for real-world usage.

Chapter 4 – System Design

4.1 ER Diagram / Database Design



The database design of the “First Park Then Pay” parking management system is structured to ensure efficient data flow, easy retrieval, and accurate billing. The Entity-Relationship (ER) diagram represents the core entities of the system—**Vehicle Types**, **Parking Blocks/Slots**, and **Vehicle Parking Records**—and shows how they are interlinked for smooth operations. Each table is designed with a clear purpose and contains attributes required for real-time tracking, billing calculations, and administration.

1. Vehicle Types (tbl_vehicletypes)

This entity stores information about different types of vehicles supported by the parking system, such as two-wheelers, cars, vans, or commercial vehicles. Each vehicle type is associated with a specific hourly rate used for billing.

Key Attributes:

- **VehicleId** – Primary key for each vehicle category
- **Name** – Type of vehicle (e.g., Car, Bike)
- **PerHourCharge** – Hourly parking rate
- **TotalSlots** – Total number of slots assigned to this vehicle category
- **IsEnabled / IsDeleted** – Operational flags
- **CreatedDate / ModifiedDate** – Timestamp logs

This table ensures that parking charges and slot allocations remain dynamic and easily manageable by administrators.

2. Vehicle Blocks / Parking Slots (tbl_vehicleblocks)

This entity represents the physical allocation of parking infrastructure. Each block may contain multiple slots and is linked to a specific vehicle type for better organization.

Key Attributes:

- **BlockId** – Unique identifier for each block
- **Name** – Block name or code (e.g., A1, B2)
- **SlotNumber** – Slot position inside the block
- **VehicleId** – Foreign key linking to **tbl_vehicletypes**
- **OwnerId** – User who created/owns the block
- **IsDeleted, CreatedDate, ModifiedDate** – Management flags and audit fields

This structure helps the system maintain real-time slot availability and supports scalability for future expansion.

3. Vehicle Parking (tbl_vehicleparking)

This is the most critical table in the system as it records every parking transaction. It tracks when a vehicle enters and leaves, calculates total hours, and determines charges based on vehicle type.

Key Attributes:

- **ParkingId** – Unique identifier for each parking session
- **Name** – Customer name
- **MobileNo / VehicleNo** – Basic customer and vehicle details
- **CheckIn / CheckOut** – Timestamp-based entry and exit
- **TotalHours** – Duration of parking
- **PerHourCharge** – Charge retrieved from vehicle type
- **TotalCharges** – Automatically calculated total payable amount
- **TransactionId** – Payment reference
- **IsPaid / PaidThrough / IsSubmitted** – Payment status fields
- **BlockName / SlotNo** – Exact location of parking
- **VehicleTypeId / VehicleTypeName** – Linked vehicle category
- **Comments** – Additional notes for admin

This entity ensures transparency, supports “Park first, pay later,” and maintains complete transaction history.

Relationships Between Entities

- **Vehicle Types ↔ Vehicle Blocks:** A single vehicle type can have multiple blocks and slots assigned to it. (One-to-Many)
- **Vehicle Types ↔ Vehicle Parking:** Each parking record is linked to exactly one vehicle type for billing and reporting. (One-to-Many)
- **Vehicle Blocks ↔ Vehicle Parking:** Each parking transaction references the specific block and slot used. (Logical Relationship)

These relationships ensure consistent data flow from slot allocation to final billing.

Summary

The ER diagram is designed to simplify parking operations by dividing data into logical entities. This modular structure supports real-time tracking, accurate billing, easy administration, and scalability for future enhancements such as IoT-based slot detection or mobile app integration.

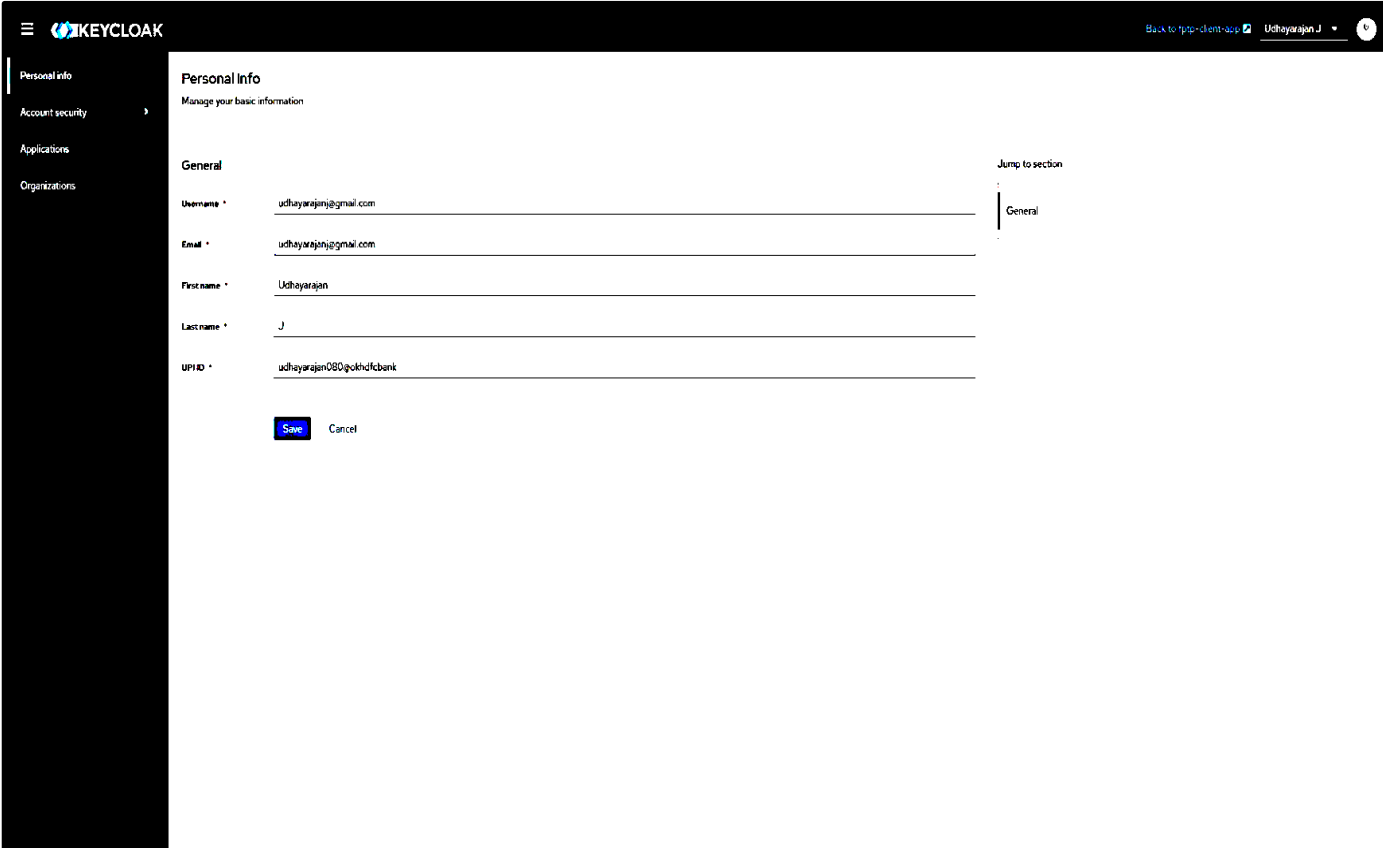
4.2 User Interface Design

The User Interface (UI) design of the parking management system is structured to offer a clean, intuitive, and user-friendly experience for both customers and administrators. The system follows a modern design approach that emphasizes simplicity, accessibility, and smooth navigation. Since authentication is handled by Key cloak, the UI integrates seamlessly with Key cloak's user management screens, ensuring a secure and consistent login experience.

The screenshot shows the Key cloak **Personal Info** page, which is part of the security framework used for identity and user profile management. This interface allows users to manage their profile information, including:

- **Username**
- **Email**
- **First Name and Last Name**
- **UPID (User Profile Identifier)**

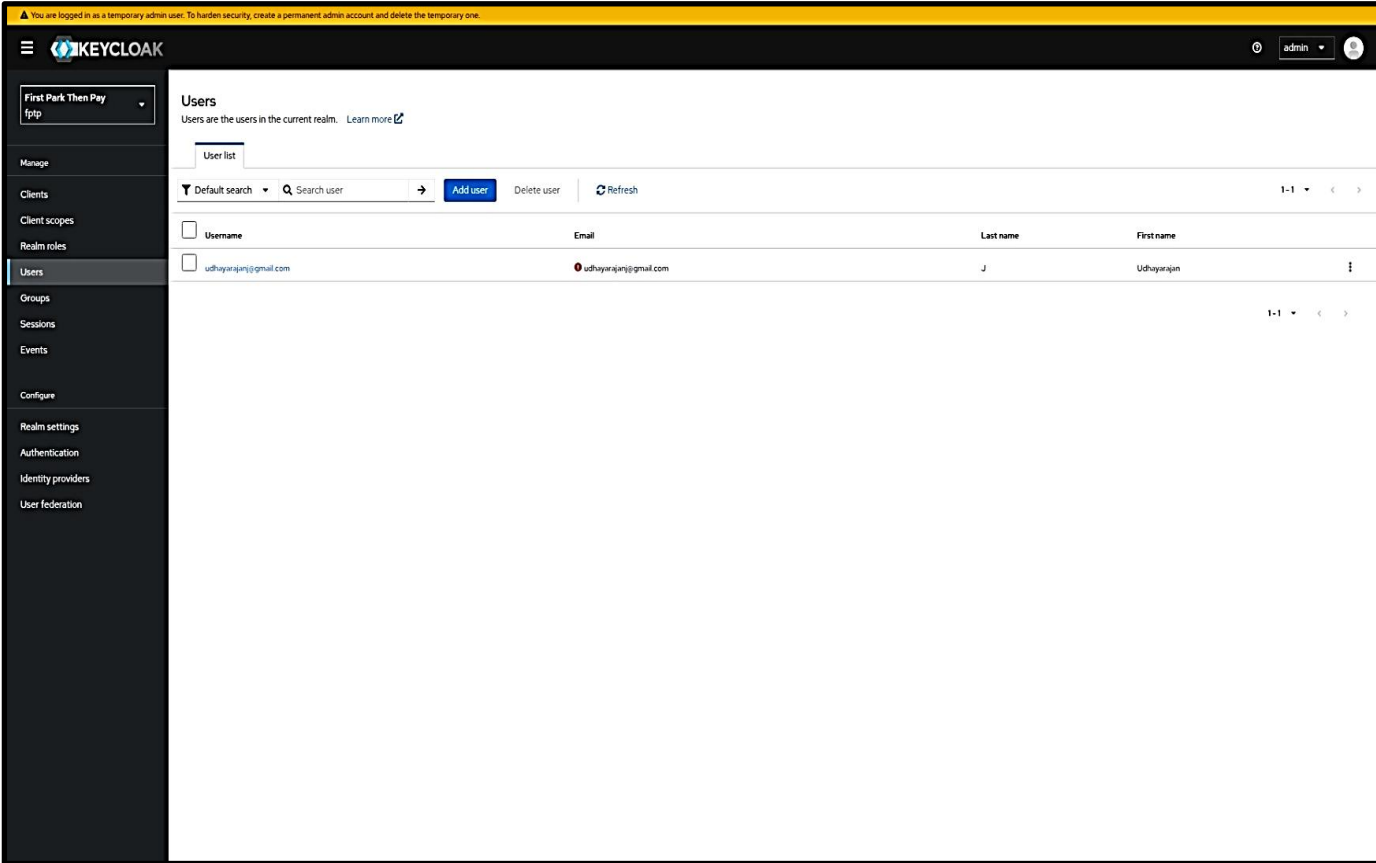
User Personal Information



The screenshot displays the Keycloak 'Personal Info' page. On the left is a dark sidebar with navigation links: 'Personal info' (active), 'Account security', 'Applications', and 'Organizations'. The main content area is titled 'Personal Info' with the subtitle 'Manage your basic information'. It features a 'General' section with five input fields: 'Username' (udhayarajin@gmail.com), 'Email' (udhayarajin@gmail.com), 'First name' (Udhayarajan), 'Last name' (J), and 'UPID' (udhayarajin080gokhdfcbank). A 'Jump to section' dropdown menu is set to 'General'. At the bottom are 'Save' and 'Cancel' buttons. The top right of the page shows a 'Back to top-level app' link, the user's name 'Udhayarajan J', and a profile icon.

The User Management interface in Keycloak serves as the administrative dashboard for handling all user-related operations within the “First Park Then Pay” parking management system. This screen is primarily used by administrators to create, update, monitor, and manage user accounts that interact with the application. Since Keycloak is the centralized identity and access management (IAM) solution, all user authentication, role assignments, and credential handling are controlled here to ensure strong security and consistency across the platform.

Admin Manage All User Details



The screenshot displays the Keycloak Admin Console interface. At the top, a yellow warning banner states: "You are logged in as a temporary admin user. To harden security, create a permanent admin account and delete the temporary one." The main header shows the Keycloak logo and the user "admin". The left sidebar contains a navigation menu with options: Manage, Clients, Client scopes, Realm roles, Users (selected), Groups, Sessions, Events, Configure, Realm settings, Authentication, Identity providers, and User federation. The "Users" section is active, showing a "User list" tab. Below the tab is a search bar with "Default search" and "Search user" options, and buttons for "Add user", "Delete user", and "Refresh". A table lists the users, with columns for Username, Email, Last name, and First name. One user is listed: "udhayanjan@gmail.com" with email "udhayanjan@gmail.com", last name "J", and first name "Udhayanjan". The table has a pagination control showing "1-1".

Username	Email	Last name	First name
udhayanjan@gmail.com	udhayanjan@gmail.com	J	Udhayanjan

Dashboard & Reporting view

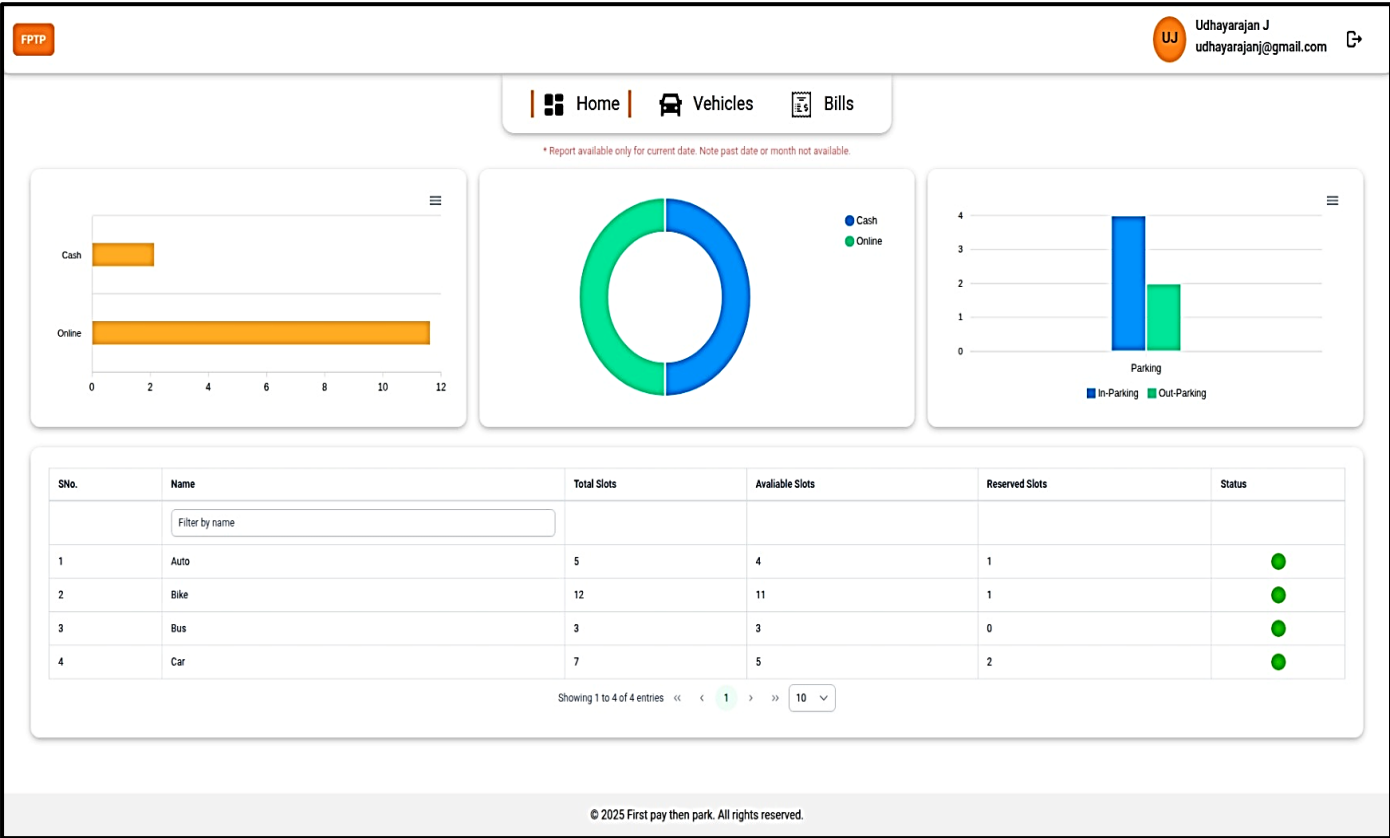
The **Dashboard & Reporting View** is the central monitoring interface of the *First Pay Then Park* (FPTP) system. It provides administrators with a real-time overview of parking-related activities, payment statistics, and slot availability. The interface is designed to be clean, responsive, and visually informative so that administrators can quickly understand system performance at a glance.

The dashboard consists of three major visual components:

- 1. **Payment Summary Charts**
- 2. **Vehicle Movement Chart (In-Parking vs Out-Parking)**
- 3. **Slot Availability Table**

These components collectively help administrators track daily usage trends, customer behaviour, and parking occupancy.

Dashboard View



Manage vehicle types

The **Manage Vehicle Types** module allows administrators to configure, update, and maintain all vehicle categories supported by the *First Pay Then Park (FPTP)* system. This module is an essential part of the parking management workflow because every billing calculation, slot allocation, and booking action depends on the proper definition of each vehicle type.

The interface is designed to provide a clear, structured, and easy-to-use management panel where administrators can add new vehicle categories, modify existing ones, and control their active/inactive status.

This module is responsible for maintaining the master data related to vehicles. It ensures:

- **Consistent pricing** for each vehicle type.
- **Accurate mapping of slot counts** in the parking premises.
- **Centralized control** over enabling or disabling vehicle categories.
- **Updated timestamp records** for auditing and tracking updates.

Manage Vehicle Details

FPTP

UJ

Udhayarajan J
udhayarajanj@gmail.com

Home

Vehicles

Bills

Add

SNo.	Name	Per/Hour	BlockName	Date	Slots	Status	
	Filter by name			Filter by date			
1	Auto	₹15.00	A	29-11-2025 04:56:09 PM	5		
2	Bike	₹10.00	BK	29-11-2025 04:55:38 PM	12		
3	Bus	₹510.00	B	29-11-2025 04:52:49 PM	3		
4	Car	₹120.00	C	29-11-2025 04:52:31 PM	7		

Showing 1 to 4 of 4 entries

<<

<

1

>

>>

10

© 2025 First pay then park. All rights reserved.

Manage billing and payments view

The *Manage Billing and Payments* module serves as a core functional component of the parking management system, enabling administrators and operators to effectively monitor vehicle entries, compute applicable charges, and manage payment settlements in an organized and transparent manner. This module integrates key operational data—such as check-in/check-out times, vehicle category, block allocation, and payment mode—to generate accurate and real-time billing information. Its central objective is to streamline the end-to-end parking transaction process while maintaining clarity, accountability, and user convenience.

At the operational level, the module provides a unified interface where users can view all active and completed parking transactions. Each record includes essential details such as vehicle number, owner or driver information, mobile number, check-in/check-out timestamps, parking duration, total hours used, block and slot allocation, and the corresponding per-hour charges. This comprehensive visibility helps operators validate vehicle activity, confirm slot usage, and ensure that the correct tariff is applied based on the selected vehicle type. The system also dynamically calculates total hours and the associated fee, reducing manual effort and eliminating errors commonly seen in paper-based or loosely managed systems.

The module also supports multiple payment modes, including cash, online payments, and pending statuses. It allows operators to instantly update payment information, mark dues as cleared, and verify whether the required keys have been submitted. Visual indicators—such as colored status markers for paid and unpaid transactions—enhance usability and reduce the likelihood of oversight during busy operational hours. This visual status tracking promotes accountability and assists staff in quickly identifying and resolving discrepancies.

Another important purpose of this module is to maintain historical transaction logs. The administrator can review previous records, track recurring users, and reconcile revenue reports. The filtering capabilities for vehicle number, owner name, mobile number, and dates further facilitate quick searches and auditing. Such traceability is vital for security, financial review, customer inquiries, and operational optimization.

From a management perspective, the module ensures seamless communication between various components of the parking system, such as vehicle entry, slot allocation, billing computation, and payment tracking. It reduces manual dependency, speeds up customer

handling time, and delivers consistent billing accuracy. By centralizing all relevant transactional details in a single interface, the module supports better decision-making, improved operational efficiency, and enhanced service quality.

Overall, the *Manage Billing and Payments* module is designed to automate and simplify the financial and administrative workflow of the parking management system. It ensures transparency, accuracy, and reliability while providing a user-friendly dashboard for operators and administrators to manage all payment-related activities efficiently.

Manage Billing Details

FPTP

UJ Udhayarajan J udhayarajanj@gmail.com

Home Vehicles Bills

Vehicle No.
Enter vehicle number

Owner Name (optional)
Enter owner name

Mobile Number (optional)
Enter mobile number

Vehicle type
- Select -

Check In
-

Block Name
Select vehicle type then Check whether available slots are not ?

Slot No.
Select vehicle type then Check whether available slots are not ?

Per/Hour Charge.
Select vehicle type.

Is key submitted ?

SNo.	Vehicle No.	Name	Mobile No.	Check In	Check Out	Total Hours	Vehicle Type.	Block Name	Slot No.	Paid	Payment Mode	Key Status	
	Filter by vehicle number	Filter by name	Filter by mobile number	Filter by check in date	Filter by check out date		- Select -			- Select -	- Select -	- Select -	
1	MH15K9989	-	-	29-11-2025 05:00:07 PM		00:03:17	Car	C	1		-		
2	TN78UJ7654	-	-	29-11-2025 04:59:35 PM		00:05:49	Auto	A	1		-		
3	TN58OL0987	-	-	29-11-2025 04:59:04 PM	29-11-2025 05:00:29 PM	00:01:22	Bus	B	1		Online		
4	TN67YH8989	-	-	29-11-2025 04:58:50 PM		00:06:34	Car	C	2		-		
5	MH15K9989	-	-	29-11-2025 04:58:38 PM	29-11-2025 04:59:45 PM	00:01:04	Car	C	1		Cash		
6	MH14LK0511	Udhayarajan J	8248748016	29-11-2025 04:56:52 PM		00:04:32	Bike	BK	1		-		

Showing 1 to 6 of 6 entries < 1 > 10

© 2025 First pay then park. All rights reserved.

35

Chapter 5 – Implementation

5.1 Coding Environment & Algorithms

The implementation of the “**First Pay Then Park**” (FPTP) system is built using a robust and modern technology stack chosen to ensure reliability, scalability, and long-term maintainability. The architecture follows a clean separation between the frontend, backend, and security layers, allowing the application to operate efficiently while also supporting future enhancements.

Backend Environment – Java 21 with Spring Boot

The backend is implemented using **Java 21**, the latest Long-Term Support (LTS) version. This version introduces improved performance, enhanced security, and modern language features such as pattern matching and record classes, all of which contribute to cleaner and more efficient code.

For the application framework, **Spring Boot** is utilized. It simplifies backend development by offering built-in tools for configuration, dependency management, and API development. Spring Boot enables the creation of RESTful APIs that handle the complete operational flow of the application—from business logic to database communication.

Core Features Implemented Using Spring Boot

- **REST API Development:** All backend operations, such as registering vehicles, managing slots, calculating charges, and updating payments, are exposed through RESTful endpoints.
- **Dependency Injection:** Spring’s IoC (Inversion of Control) container ensures cleaner code by automatically managing dependency lifecycles.
- **Exception Handling:** Centralized exception handling provides meaningful error messages and ensures application stability during runtime failures.
- **Spring Data JPA:** This enables seamless interaction with the database through repository interfaces, minimizing boilerplate code and improving data management efficiency.

- **Service-Oriented Architecture:** Business logic is separated into service layers, making the system modular, testable, and easy to maintain.

Overall, the Java-Spring Boot backend handles all data processing, validations, authentication checks, and computations required by the parking management workflow.

Frontend Environment – Angular

The user interface of the system is built using **Angular**, a powerful frontend framework that supports component-based architecture and reactive programming. Angular ensures that the UI remains responsive, organized, and easy for users to navigate.

How Angular Works in the Project

- It provides a **dynamic UI** based on reusable components such as forms, tables, dialogs, and dashboards.
- Angular communicates with the backend using **HTTP Client**, sending GET, POST, PUT, and DELETE requests as needed.
- Forms are used extensively for **vehicle type creation, slot booking, check-in and check-out actions, billing entries, and payment updates.**
- Angular services act as data handlers, storing temporary application data like user authentication tokens.

This component-based structure allows rapid loading, smooth interactions, and high maintainability.

Interaction Between Angular and Spring Boot

The entire system follows a standard client-server architecture. The communication flow works as follows:

1. **A user interacts with the Angular interface**, for example, by checking in a vehicle or processing a payment.
2. Angular sends an **HTTP request** to a corresponding Spring Boot endpoint.
3. **Spring Boot processes the request**, validates data, and interacts with the MySQL database through Spring Data JPA.

4. The backend returns a **JSON response** containing processed data, success messages, or error details.
5. Angular receives the response and **updates the UI** accordingly—for example, refreshing the vehicle list or displaying the generated bill.

This seamless interaction ensures real-time data flow between the user interface and the database.

JWT-Based Authentication Using Key cloak

Security is implemented using **Key cloak**, an identity and access management tool.

The authentication process works as follows:

- Users log in through Key cloak's authentication screen.
- Key cloak validates the credentials and issues a **JWT (JSON Web Token)**.
- Angular stores the JWT in local or session storage.
- For every API request, Angular attaches the token in the Authorization header:
Authorization: Bearer <token>
- Spring Boot validates this token using Key cloak adapters before processing any request.

This ensures secure access control, preventing unauthorized users from interacting with the system.

Parking Charge Calculation Algorithm

One of the core algorithms in the system is the parking fee calculation. The total charge is based on the parking duration and the predefined rate for each vehicle type.

Formula

`total Charge = hours Parked × ratePerHour`

Algorithm Steps

1. Capture and record the vehicle's **entry time**.
2. Capture the **exit time** during checkout.
3. Compute total time spent using timestamp difference.
4. Identify the vehicle type and fetch its **hourly rate**.
5. Multiply total hours by the hourly rate.
6. Generate the billing record and update the payment status (Paid, Online, Pending, etc.).

This simple yet effective algorithm ensures accurate and automated billing without manual intervention.

5.2 Module Description

1. User Management Module

The User Management module is responsible for handling authentication, authorization, and role-based access to the system. This module is fully integrated with **Key cloak**, a dedicated Identity and Access Management (IAM) tool that offers secure user login, centralized credential storage, and token-based authentication.

Through Key cloak, the system supports **role-based access control (RBAC)**, ensuring that users only access the features relevant to their responsibilities. Two primary roles are defined:

- **Admin** – Has full control over vehicle types, slot configuration, billing operations, user monitoring, and overall system management.
- **Customer** – Can view vehicle information, make bookings, and complete payments as permitted by the system.

By delegating security to Key cloak, the system benefits from industry-standard protocols such as OAuth2 and OpenID Connect, along with secure JWT token-based API communication. This results in a highly secure and well-structured authentication workflow.

2. Vehicle Management Module

The Vehicle Management module handles everything related to vehicle categories and parking infrastructure configuration. It allows administrators to:

- **Create and manage vehicle types** such as car, bike, auto, and bus.
- **Set hourly parking charges** for each category, enabling dynamic pricing based on vehicle type.
- **Configure parking blocks and slots**, allowing the system to map out the physical layout of the parking facility.

This module ensures that all vehicle-related information is well-organized and easily accessible. The configuration capabilities help administrators scale the parking layout as needed and adjust pricing to reflect business requirements. Since billing depends heavily on

vehicle type and slot allocation, this module acts as the foundation for accurate charge calculation and efficient space management.

3. Billing and Payment Module

The Billing and Payment module manages the entire financial workflow of the system. Its primary function is to automatically compute the parking fee based on the duration of stay and the hourly rate configured for the vehicle type.

Key functionalities include:

- **Automated parking duration calculation**, based on recorded check-in and check-out timestamps.
- **Support for multiple payment modes**, including **Cash** and **Online** entries.
- **Validation of payment status**, ensuring that vehicles cannot exist without clearing dues.

The system calculates charges using a time-based billing algorithm and updates records in real-time. This module eliminates manual calculations, reduces errors, and enhances transparency for both administrators and users. It also serves as a centralized financial tracking system, maintaining a detailed history of all bills and payment statuses.

4. Dashboard Module

The Dashboard Module provides a real-time summary of the system's operations using visual data and summarized statistics. It acts as an analytical control centre for administrators.

Core features include:

- **Real-time parking data**, showing active check-ins, available slots, and current occupancy.
- **Payment summary**, displaying paid and pending transactions, total revenue, and mode-wise payment distribution.
- **Slot usage statistics**, helping management understand peak hours, slot demand, and overall parking efficiency.

This module enables informed decision-making by providing a comprehensive overview of system performance. The dashboard's data insights help administrators optimize parking operations, plan for future capacity, and monitor overall system activity.

Chapter 6 – Conclusion & Future Enhancements

6.1 Conclusion

The “First Park Then Pay” (FPTP) system has been successfully designed and implemented as a comprehensive solution for modern parking space management. The system addresses long-standing challenges faced by traditional parking environments, such as manual record-keeping, inconsistent billing practices, inefficient slot allocation, and lack of real-time monitoring. By leveraging modern technologies such as **Spring Boot**, **Angular**, **Key cloak**, and **Docker**, the platform delivers an efficient, secure, and scalable solution suitable for commercial parking sites, institutions, and public infrastructure.

One of the most significant achievements of the FPTP system is the automation of essential parking operations. Through a streamlined workflow, the system captures vehicle entry and exit data, calculates the total parking duration, generates bills automatically, and validates payment status before vehicle exit. This end-to-end automation greatly reduces human error, minimizes financial discrepancies, and ensures transparent billing for all users. The incorporation of dynamic rate configuration based on vehicle type—such as bike, auto, car, or bus—further enhances operational flexibility and revenue accuracy for administrators.

The system’s modular design contributes significantly to its robustness. Each module—User Management, Vehicle Management, Billing and Payment, and Dashboard—operates both independently and cohesively, ensuring maintainability and ease of future scalability. The User Management module, powered by Keycloak, guarantees secure authentication and role-based access. Administrators and customers interact with the system according to their privileges, ensuring data safety and system integrity. The Vehicle Management and Billing modules work hand in hand, enabling accurate calculation of charges while maintaining complete visibility into slot usage and vehicle movements. The Dashboard module adds an analytical layer, offering real-time insights into occupancy levels, revenue trends, and operational performance.

From a technical standpoint, the system embraces a modern architecture aligned with current industry standards. The backend developed using **Java 21** and **Spring Boot** enables clean code, strong performance, and efficient API communication. The frontend built with **Angular** delivers a responsive, component-driven interface that ensures an intuitive user experience. Security integration through **Key cloak** ensures compliance with authentication protocols such

as OAuth2 and OpenID Connect, offering reliable protection for user data. The deployment with **Docker containers** makes the system portable, environment-consistent, and ready for large-scale distribution.

Overall, the “First Park Then Pay” system provides a powerful foundation for smart parking automation. It improves customer experience, reduces manual dependencies, enhances financial accuracy, and equips administrators with the tools required for seamless operational oversight. With its clean architecture, strong security measures, and flexible modular design, the system stands as a future-ready platform capable of supporting expansions and technological integrations for years to come.

6.2 Future Enhancements

Although the system currently provides a robust set of core features, several enhancements can further improve functionality, efficiency, and user convenience. As technology continues to evolve, the system can be expanded to incorporate advanced capabilities that would transform it into a fully automated smart parking ecosystem.

1. IoT-Based Slot Automation

One of the most promising future enhancements is the integration of **Internet of Things (IoT) sensors** for automated slot detection. By installing IR sensors, ultrasonic sensors, RFID readers, or smart cameras in parking slots, the system can automatically detect slot availability without requiring manual check-in or check-out entries.

Benefits include:

- Real-time automatic updates of slot occupancy
- Elimination of manual errors in slot allocation
- Improved traffic flow and reduced congestion
- Enhanced customer satisfaction through live availability displays

IoT-based automation would make the system highly intelligent, enabling automated guidance systems, digital signboards, and mobile app notifications showing real-time parking availability.

2. Mobile Application Development

To increase accessibility and user convenience, a dedicated **mobile app** can be developed for Android and iOS platforms. The mobile app would allow customers to:

- View parking availability in real time
- Pre-book slots (if allowed by policy)
- Receive digital invoices and payment reminders
- Maintain booking history
- Navigate to available slots using integrated maps
- Perform online payments directly from the app

Administrators, on the other hand, could use the app for quick vehicle searches, instant updating of slot status, and real-time monitoring of revenue and occupancy. A mobile app would significantly strengthen the system's usability and reach.

3. Online Payment Gateway Integration

Currently, the system supports cash and manual online entries. Integrating a **payment gateway** such as Razor pay, Stripe, Paytm, or UPI-based platforms would enable customers to make immediate digital payments at checkout.

This enhancement would offer:

- Secure online transactions
- Automatic receipt generation
- Reduced cash handling
- Faster and more convenient payments
- Improved customer satisfaction

Payment gateway integration would also simplify revenue tracking and accounting for administrators, making the system more professional and business-friendly.

4. AI-Based Parking Prediction and Analytics

With the growing capabilities of artificial intelligence, the system can be enhanced to include predictive analytics features. Using historical data on occupancy, time-of-day patterns, and event-driven demand, an **AI model** can predict:

- Peak parking times
- Demand for specific vehicle types
- Slot availability in the next hour
- Seasonal or weekly usage trends

AI-driven prediction would allow better resource planning, shift management, and dynamic pricing strategies. This would elevate the parking system from being simply operational to being a strategic decision-support tool for administrators.

5. License Plate Recognition (LPR) Integration

Another potential enhancement is the integration of camera-based **Automatic License Plate Recognition** (ALPR) technology. With LPR, the system can automatically detect vehicle numbers during entry and exit without manual input.

Advantages include:

- Faster check-in/check-out
- Reduced human intervention
- Higher accuracy in vehicle logging
- Enhanced security and auditability

This enhancement would greatly reduce the operational workload and improve throughput during peak hours.

6. Cloud Deployment and Multi-Branch Support

As the system evolves, deploying it on cloud platforms such as AWS, Azure, or Google Cloud can enable:

- High availability
- Scalability for multiple parking facilities
- Centralized data management
- Disaster recovery and backup systems

Cloud deployment combined with multi-branch support can enable large enterprises or city-level parking networks to use a single unified system for multiple locations.

Chapter 7 – Bibliography

7.1 References

- **Angular Framework Documentation** Official documentation for Angular, covering components, services, modules, forms, and HTTP client usage.

Available at: <https://angular.io/docs>

- **Node.js Official Documentation** Comprehensive reference for Node.js runtime, packages, APIs, and development tools.

Available at: <https://nodejs.org/en/docs>

- **Java 21 Documentation (Oracle)** Official Oracle documentation for Java 21, including language features, APIs, libraries, and development tools.

Available at: <https://docs.oracle.com/en/java/>

- **Spring Boot Framework Documentation** Detailed documentation on Spring Boot features, configuration, dependency management, and REST API development.

Available at: <https://docs.spring.io/spring-boot/docs/current/reference/html/>

- **Key cloak Documentation** Official guide for Key cloak authentication, authorization, user federation, and JWT-based security.

Available at: <https://www.keycloak.org/documentation>

- **MySQL Database Documentation** Reference for MySQL SQL syntax, database design, administration, and connector tools.

Available at: <https://dev.mysql.com/doc/>

- **Docker Documentation** on Docker containers, images, Compose, deployment, and environment setup.

Available at: <https://docs.docker.com/>

- **Visual Studio Code Documentation** Official documentation for VS Code editor features, extensions, debugging tools, and shortcuts.

Available at: <https://code.visualstudio.com/docs>

- **IntelliJ IDEA Community Edition Documentation** Reference for IntelliJ project setup, plugin management, debugging, and Java development tools.

Available at: <https://www.jetbrains.com/idea/documentation/>

- **Project Source Code (GitHub Repository)** Repository containing the complete source code, modules, and documentation for the FPTP (First Pay Then Park) system.

Available at: <https://github.com/UdhayarajanJ/2202561-MCA-2025-2027.git>